# tasklist.md — BootPulse Phase 1 MVP Execution Plan

**Source document:** PRD.md — BootPulse v1.0 (Phase 1 MVP)
**Execution owner:** AI Coding Agent
**Plan owner:** Senior Agile Scrum Master / Technical Project Manager
**Scope:** Phase 1 Core MVP only. Phase 2 items are isolated in the final **Deferred** section and must not be built.
**Legend:** - [ ] = atomic task · **Verification Gate** = blocking checkpoint before proceeding to the next step.

> **Rule for the coding agent:** Do not start a module until the previous module's Verification Gate passes. Do not implement anything listed under *Deferred*. If a task is marked `N/A — CONFIRMED`, do not invent a substitute feature.

---

# Module 1 — Environment & Secret Key Setup (`.env.local`)

## 1.1 Project Scaffold

- ☐ Initialise the Next.js App Router project with TypeScript: `npx create-next-app@latest bootpulse --typescript --app --eslint --tailwind --src-dir --import-alias "@/*"`.
- ☐ Confirm `src/app/` exists and `src/app/layout.tsx` renders a root layout.
- ☐ Set `"strict": true` in `tsconfig.json`.
- ☐ Add `"noUncheckedIndexedAccess": true` and `"exactOptionalPropertyTypes": true` to `tsconfig.json` compiler options.
- ☐ Confirm `paths` alias `@/*` resolves to `src/*`.
- ☐ Install Prettier and `prettier-plugin-tailwindcss` as dev dependencies.
- ☐ Add `.prettierrc` with `singleQuote: true`, `semi: true`, `printWidth: 100`.
- ☐ Add npm scripts: `dev`, `build`, `start`, `lint`, `typecheck` (`tsc --noEmit`), `format`.
- ☐ Initialise git repository and create the initial commit.

## 1.2 Ignore Rules

- ☐ Confirm `.gitignore` includes `.env`, `.env.local`, `.env.*.local`, `node_modules`, `.next`, `coverage`.

- ☐
  Confirm `.gitignore` explicitly does **not** ignore `.env.example`.

## 1.3 Environment Key Templates

- ☐
  Create `.env.example` containing key **names only** (no values) for: `DATABASE_URL`, `DATABASE_URL_UNPOOLED`.
- ☐
  Add to `.env.example`: `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`, `CLERK_SECRET_KEY`, `CLERK_WEBHOOK_SIGNING_SECRET`, `NEXT_PUBLIC_CLERK_SIGN_IN_URL`, `NEXT_PUBLIC_CLERK_SIGN_UP_URL`, `NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL`, `NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL`.
- ☐
  Add to `.env.example`: `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`, `STRIPE_PRICE_STARTER_MONTHLY`, `STRIPE_PRICE_GROWTH_MONTHLY`, `STRIPE_PRICE_SCALE_MONTHLY`.
- ☐
  Add to `.env.example`: `NEXT_PUBLIC_APP_URL`.
- ☐
  Add commented-out `GEMINI_API_KEY`, `IMAGEKIT_PUBLIC_KEY`, `IMAGEKIT_PRIVATE_KEY`, `IMAGEKIT_URL_ENDPOINT` with the note `# NOT APPLICABLE in Phase 1`.
- ☐
  Create `.env.local` locally and populate it with placeholder/real test-mode values.
- ☐
  Confirm `.env.local` is not tracked by `git status`.

## 1.4 Environment Variable Validation

- ☐
  Install `zod`.
- ☐
  Create `src/lib/env/server.ts` exporting a Zod schema for all server-only variables.
- ☐
  Create `src/lib/env/client.ts` exporting a Zod schema for `NEXT_PUBLIC_*` variables only.
- ☐
  Ensure `server.ts` throws a descriptive error at import time if a required variable is missing or malformed.
- ☐
  Ensure `server.ts` is never imported from a file containing `'use client'`.
- ☐
  Add a CI-safe guard so `server.ts` fails fast if `NEXT_PUBLIC_` is prefixed onto a secret key name.

## 1.5 Secret Scanning

- ☐
  Add a `gitleaks` configuration file (`.gitleaks.toml`) with default rules enabled.
- ☐
  Add a GitHub Actions workflow `.github/workflows/ci.yml` running `gitleaks detect --no-git-history`.
- ☐
  Add `npm run lint` and `npm run typecheck` to the same CI workflow.
- ☐
  Confirm CI fails the build if a secret pattern is detected.

## ✅ Verification Gate — Module 1

- ☐
  Run `npm run build` and confirm **zero TypeScript errors**.
- ☐
  Run `npm run lint` and confirm **zero errors**.
- ☐
  Run `npm run typecheck` and confirm **zero errors**.
- ☐
  Run `gitleaks detect` and confirm **zero findings**.
- ☐
  Confirm no secret value appears anywhere in tracked source files (`git grep` for `sk_test`, `pk_test`, `whsec`, `postgresql://`).

---

# Module 2 — Database Schema & ORM Provisioning (Drizzle / PostgreSQL)

## 2.1 Drizzle & Driver Setup

- ☐
  Install `drizzle-orm`, `drizzle-kit`, and `postgres` (or `@neondatabase/serverless`).
- ☐
  Create `drizzle.config.ts` pointing at `src/db/schema.ts`, reading `DATABASE_URL_UNPOOLED` for migrations.
- ☐
  Create `src/db/index.ts` exporting a pooled Drizzle client using `DATABASE_URL`.
- ☐
  Create a separate `src/db/migrate.ts` client using `DATABASE_URL_UNPOOLED` for migrations only.
- ☐
  Add npm scripts: `db:generate`, `db:migrate`, `db:push`, `db:seed`, `db:studio`.

- ☐
  Confirm `src/db/index.ts` never uses a superuser or BYPASSRLS role.

## 2.2 Enums (`src/db/schema.ts`)

- ☐
  Define `planTierEnum` — `['starter', 'growth', 'scale']`.
- ☐
  Define `subStatusEnum` — `['trialing', 'active', 'past_due', 'canceled', 'incomplete']`.
- ☐
  Define `memberRoleEnum` — `['owner', 'editor', 'viewer']`.
- ☐
  Define `memberStatusEnum` — `['invited', 'active', 'revoked']`.
- ☐
  Define `itemTypeEnum` — `['revenue', 'expense']`.
- ☐
  Define `recurrenceEnum` — `['one_off', 'monthly', 'custom']`.
- ☐
  Define `scenarioKindEnum` — `['baseline', 'scenario']`.
- ☐
  Define `assumptionTypeEnum` — `['add_expense', 'remove_expense', 'change_revenue', 'one_off_cost', 'one_off_gain']`.
- ☐
  Define `decisionStatusEnum` — `['considering', 'committed', 'rejected', 'revisited']`.
- ☐
  Define `milestoneStateEnum` — `['planned', 'achieved', 'missed']`.
- ☐
  Define `warningLevelEnum` — `['critical', 'warning', 'healthy']`.

## 2.3 Tenant & Identity Tables

- ☐
  Define `users` table with UUID PK, `clerkUserId` (unique, not null), `displayName`, `contactEmail`, `planTier`, `trialEndsAt`, `consentPrivacyAt`, `consentTermsAt`, `marketingOptIn` (default `false`), `createdAt`, `updatedAt`.
- ☐
  Define `businesses` table with UUID PK, `ownerUserId` FK → `users.id` **ON DELETE CASCADE**, `name`, `currencyCode` (default `'USD'`), `startingCashMinor`, `startMonth`, `forecastMonths`, `burnWindowMonths`, `criticalThresholdMonths`, `warningThresholdMonths`, `stripeCustomerId`, timestamps.
- ☐
  Add index `businesses_owner_idx` on `businesses.ownerUserId`.
- ☐

Define `team_members` table with UUID PK, `businessId` FK → `businesses.id` **ON DELETE CASCADE**, `userId` FK → `users.id` **ON DELETE CASCADE** (nullable), `invitedEmail`, `role`, `status`, `invitedAt`, `acceptedAt`, `createdAt`.

- ☐
  Add index `team_members_business_idx` on `team_members.businessId`.

## 2.4 Financial Data Tables

- ☐
  Define `categories` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `name`, `itemType`, `colorToken`, `isRecurring`, `sortOrder`, `createdAt`.
- ☐
  Add index `categories_business_idx`.
- ☐
  Define `revenue_items` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `categoryId` FK → `categories.id` **ON DELETE SET NULL**, `label`, `amountMinor`, `recurrence`, `startMonth`, `endMonth`, `growthRateBps` (default `0`), `notes`, timestamps.
- ☐
  Add index `revenue_items_business_idx`.
- ☐
  Define `expense_items` table with the same shape as `revenue_items` (distinct table name and index `expense_items_business_idx`).
- ☐
  Define `scenarios` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `name`, `kind`, `description`, `horizonMonths`, `createdByUserId` FK → `users.id` **ON DELETE SET NULL**, `isArchived`, timestamps.
- ☐
  Add index `scenarios_business_idx`.
- ☐
  Define `scenario_assumptions` table with UUID PK, `scenarioId` FK → `scenarios.id` **ON DELETE CASCADE**, `assumptionType`, `label`, `amountMinor`, `startMonth`, `endMonth`, `isEnabled`, `sortOrder`, `createdAt`.
- ☐
  Add index `scenario_assumptions_scenario_idx`.
- ☐
  Define `monthly_summaries` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `scenarioId` FK → `scenarios.id` **ON DELETE CASCADE**, `month`, `openingCashMinor`, `revenueMinor`, `expenseMinor`, `netCashFlowMinor`, `closingCashMinor`, `runwayMonths` (`numeric(6,2)`), `warningLevel`, `computedAt`.
- ☐
  Add index `monthly_summaries_business_idx`.
- ☐
  Add unique index `monthly_summaries_biz_scenario_month_uq` on (`businessId, scenarioId, month`).

## 2.5 Planning & Narrative Tables

- ☐
  Define `milestones` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `title`, `description`, `targetMonth`, `state`, timestamps.
- ☐
  Add index `milestones_business_idx`.
- ☐
  Define `decisions` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `scenarioId` FK → `scenarios.id` **ON DELETE SET NULL**, `title`, `rationale`, `estimatedMonthlyImpactMinor`, `status`, `decidedAt`, timestamps.
- ☐
  Add index `decisions_business_idx`.
- ☐
  Define `notes` table with UUID PK, `businessId` FK **ON DELETE CASCADE**, `decisionId` FK → `decisions.id` **ON DELETE CASCADE** (nullable), `month` (nullable), `body`, `createdByUserId` FK **ON DELETE SET NULL**, `createdAt`.
- ☐
  Add index `notes_business_idx`.

## 2.6 Webhook Idempotency Table

- ☐
  Define `processed_webhook_events` table with UUID PK, `eventId` (text, unique, not null), `eventType`, `processedAt` (default now), `payloadHash`.
- ☐
  Add unique index on `processed_webhook_events.eventId`.
- ☐
  Confirm this table has **no** RLS policy requiring business access (it is system-scoped; restrict to the server role only).

## 2.7 Migration Generation & Application

- ☐
  Run `npm run db:generate` and confirm a SQL migration file is produced.
- ☐
  Inspect the generated SQL and confirm every FK uses `ON DELETE CASCADE` / `SET NULL` exactly as specified.
- ☐
  Run `npm run db:migrate` against the Neon **direct** connection string.
- ☐
  Confirm all 13 tables exist in the database.

## 2.8 Row Level Security

- ☐
  Create migration `0002_rls.sql` (or a Drizzle custom migration) enabling RLS on: `users`, `businesses`, `team_members`, `categories`, `revenue_items`, `expense_items`, `monthly_summaries`, `scenarios`, `scenario_assumptions`, `milestones`, `decisions`, `notes`.
- ☐
  Implement `app_current_user_id()` SQL function resolving `request.jwt.claims ->> 'sub'` → `users.id`.
- ☐
  Implement `app_has_business_access(target_business_id uuid)` SQL function covering owner and active `team_members`.
- ☐
  Add policy `users_self_select` (select where `clerk_user_id` = JWT sub).
- ☐
  Add policy `users_self_update` (update where `clerk_user_id` = JWT sub).
- ☐
  Add policy `businesses_owner_all` (`for all` + `with check` on `owner_user_id`).
- ☐
  Add policy `businesses_member_select` (`for select` using `app_has_business_access(id)`).
- ☐
  Add `team_members_access` policy (`for all` + `with check` on `business_id`).
- ☐
  Add `categories_access` policy.
- ☐
  Add `revenue_items_access` policy.
- ☐
  Add `expense_items_access` policy.
- ☐
  Add `monthly_summaries_access` policy.
- ☐
  Add `scenarios_access` policy.
- ☐
  Add `scenario_assumptions_access` policy using a subquery through `scenarios.business_id`.
- ☐
  Add `milestones_access` policy.
- ☐
  Add `decisions_access` policy.
- ☐
  Add `notes_access` policy.
- ☐

Confirm the runtime DB role is **not** a superuser and does **not** hold BYPASSRLS.

- ☐
  Add a transactional helper `withUserContext(clerkJwt, fn)` that runs `SET LOCAL request.jwt.claims = ...` before executing queries.

## 2.9 Domain Calculation Engine

- ☐
  Define the branded type `MinorUnits = number & { __brand: 'MinorUnits' }` in `src/lib/money.ts`.
- ☐
  Implement `formatMoney(minorUnits, currency)` — the single rounding/display point.
- ☐
  Create `src/lib/calc/runway.ts` as **pure, side-effect-free** functions.
- ☐
  Implement `netCashFlow(month) = revenueTotal(month) − expenseTotal(month)`.
- ☐
  Implement `closingCash(month) = openingCash(month) + netCashFlow(month)`.
- ☐
  Implement `openingCash(month+1) = closingCash(month)` and `openingCash(first) = business.startingCashMinor`.
- ☐
  Implement `avgMonthlyBurn` = mean of `max(0, −netCashFlow(m))` over the trailing window (default 3 months).
- ☐
  Implement `runwayMonths` = `currentCash / avgMonthlyBurn` when burn > 0, otherwise the literal `"Cash-flow positive"`.
- ☐
  Implement `zeroCashMonth` = first month where `closingCash < 0`.
- ☐
  Implement `warningLevel(runwayMonths, critical, warning)` returning `critical | warning | healthy`.
- ☐
  Implement `forecastFromItems(items, startMonth, horizonMonths)` expanding recurrence (`one_off`, `monthly`, `custom`) and applying `growthRateBps`.
- ☐
  Return a `formulaTrace` object from every calculation exposing inputs + expression string for `FormulaDisclosure`.
- ☐
  Write unit tests for: cash-flow positive case, burn case, zero-cash boundary, growth rate application, and trailing-window burn.
- ☐
  Run the unit test suite and confirm all pass.

## 2.10 Fictional Seed Data

- ☐
  Create `src/db/seed.ts` using **fictional data only**.
- ☐
  Seed one demo user with `displayName: 'Priya Demo'` and `contactEmail: 'priya@example.com'`.
- ☐
  Seed one demo business `Priya Demo Co.` with a fictional `startingCashMinor`.
- ☐
  Seed fictional revenue categories (e.g. `Consulting`, `Product Sales`) and expense categories (e.g. `Software`, `Contractors`).
- ☐
  Seed 2–5 fictional revenue items and 2–5 fictional expense items.
- ☐
  Seed a baseline scenario (`kind: 'baseline'`).
- ☐
  Seed 1 fictional scenario `Hire a freelancer` with an `add_expense` assumption of `200000` minor units starting month 3.
- ☐
  Seed 2 fictional milestones and 1 fictional decision.
- ☐
  Confirm **no** real names, emails, phone numbers, addresses, or real financial figures appear in the seed file.

## ✅ Verification Gate — Module 2

- ☐
  Run `npm run db:generate` and confirm it produces no unexpected diff.
- ☐
  Run `npm run db:migrate` and confirm it completes with zero errors.
- ☐
  Connect with `psql` and confirm all 13 tables exist with the expected columns.
- ☐
  Run `select * from pg_policies;` and confirm a policy exists for every tenant table.
- ☐
  Run a query as the runtime role **without** a JWT set and confirm it returns **zero rows** from `businesses`.
- ☐
  Run the same query **with** a valid test JWT set and confirm it returns only the caller's rows.
- ☐
  Run `npm run db:seed` and confirm it completes without error.
- ☐
  Run `npm run typecheck` and confirm zero errors.

- ☐
  Run the unit test suite for `src/lib/calc/runway.ts` and confirm all pass.

---

# Module 3 — Authentication & Protected Route Middleware

## 3.1 Clerk Provisioning

- ☐
  Create a Clerk application in **development** mode.
- ☐
  Copy the publishable and secret keys into `.env.local`.
- ☐
  Install `@clerk/nextjs`.
- ☐
  Wrap the root layout in `<ClerkProvider>` in `src/app/layout.tsx`.
- ☐
  Confirm `ClerkProvider` receives no secret values via props.

## 3.2 Middleware & Route Protection

- ☐
  Create `src/middleware.ts` using `clerkMiddleware()`.
- ☐
  Mark `/app(.*)` and `/onboarding` as protected routes.
- ☐
  Mark `/`, `/pricing`, `/legal(.*)`, `/sign-in(.*)`, `/sign-up(.*)`, `/api/webhooks(.*)` as public.
- ☐
  Configure `NEXT_PUBLIC_CLERK_SIGN_IN_URL`, `NEXT_PUBLIC_CLERK_SIGN_UP_URL`, `NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL`, `NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL` in `.env.local`.
- ☐
  Confirm an unauthenticated request to `/app` redirects to `/sign-in`.

## 3.3 Auth Pages

- ☐
  Create `src/app/sign-in/[[...rest]]/page.tsx` rendering `<SignIn />`.
- ☐
  Create `src/app/sign-up/[[...rest]]/page.tsx` rendering `<SignUp />`.
- ☐
  Confirm the sign-up form supports paste and password managers (Clerk default — verify, do not disable).

- ☐
  Confirm no custom cognitive-function challenge is added to either page.

## 3.4 User Sync (Clerk → `users`)

- ☐
  Create `src/app/api/webhooks/clerk/route.ts`.
- ☐
  Verify the Clerk webhook signature using `CLERK_WEBHOOK_SIGNING_SECRET` against the **raw** body.
- ☐
  Reject with HTTP `400` on signature verification failure.
- ☐
  Handle `user.created`: insert a `users` row with `clerkUserId`, `displayName`, `contactEmail`.
- ☐
  Handle `user.created`: write `consentPrivacyAt` and `consentTermsAt` server-side from the sign-up consent checkbox state.
- ☐
  Handle `user.created`: set `marketingOptIn` from the **separate, unticked-by-default** marketing checkbox only.
- ☐
  Handle `user.updated`: update `displayName` and `contactEmail`.
- ☐
  Handle `user.deleted`: delete the `users` row (cascade removes all tenant data).
- ☐
  Record the Clerk event ID for idempotency (reuse `processed_webhook_events`).
- ☐
  Confirm consent timestamps are never back-dated and are append-only.

## 3.5 Server-Side Authorisation Helper

- ☐
  Create `src/lib/auth/requireUser.ts` that calls Clerk `auth()` and loads the matching `users` row.
- ☐
  Throw a typed `UnauthorizedError` if no Clerk session exists.
- ☐
  Create `src/lib/auth/requireBusinessAccess.ts` that accepts a `businessId` and verifies ownership or active `team_members` membership.
- ☐
  Throw a typed `ForbiddenError` if access is denied.
- ☐
  Confirm every Server Action and protected Route Handler calls one of these helpers before touching the database.

- ☐
  Confirm `businessId` is **never** trusted from client input without this check.

## 3.6 Tier Gating Helper

- ☐
  Create `src/lib/billing/tiers.ts` defining the tier → limits map (`starter`: 1 business / 6 months / 2 scenarios; `growth`: 3 / 18 / 10; `scale`: 10 / 36 / unlimited).
- ☐
  Implement `assertWithinTier(user, business, limitType)` used by all creation and horizon-extension actions.
- ☐
  Implement read-only degradation: when data exceeds the current tier limit, mark it read-only rather than deleting it.
- ☐
  Confirm the trial state maps to **Growth** limits and expiry downgrades to Starter read-only.

## 3.7 Onboarding Wizard

- ☐
  Create `src/app/onboarding/page.tsx` as a protected 4-step wizard.
- ☐
  Step 1: business name, currency (single, default USD), business start month.
- ☐
  Step 2: starting cash available today.
- ☐
  Step 3: add 2–5 revenue lines and 2–5 expense lines, pre-filled with **editable fictional examples**.
- ☐
  Step 4: confirm assumptions (trailing burn window, critical threshold, warning threshold) with a "Create my dashboard" CTA.
- ☐
  Implement the Server Action that creates the business, its categories, its items, and its **baseline scenario** in a single transaction.
- ☐
  Redirect to `/app/businesses/[businessId]` on success.
- ☐
  Confirm the wizard is fully keyboard navigable and shows no colour-only state.
- ☐
  Confirm all placeholder data is fictional.

## ✅ Verification Gate — Module 3

- ☐
  Run `npm run build` and confirm **zero TypeScript errors**.

- ☐
  Sign up as a new test user and confirm `/onboarding` is reachable.
- ☐
  Complete onboarding and confirm a `businesses` row plus a baseline `scenarios` row exist.
- ☐
  Confirm the `users` row contains non-null `consentPrivacyAt` and `consentTermsAt`.
- ☐
  Sign out and request `/app` directly — confirm redirect to `/sign-in`.
- ☐
  Sign in as a **second** test user and confirm the first user's business is not visible in the UI or the API.
- ☐
  Attempt a Server Action with the second user's session and the first user's `businessId` — confirm it throws `ForbiddenError`.
- ☐
  Run `gitleaks detect` and confirm zero findings.

---

# Module 4 — Cloud Storage & Third-Party API Integrations

> **Scope note:** The PRD explicitly marks **ImageKit** and **Gemini API** as *NOT APPLICABLE to Phase 1*. This module records that decision and covers the only third-party integration boundary actually required: the Stripe SDK client and tier→price mapping. Do not build media upload or AI summary features.

## 4.1 ImageKit — Confirmed Out of Scope

- ☐
  **N/A — CONFIRMED:** Do **not** install `imagekit` or `imagekit-javascript`.
- ☐
  **N/A — CONFIRMED:** Do **not** create any upload route, media table, or transformation helper.
- ☐
  **N/A — CONFIRMED:** Do **not** add ImageKit keys to `.env.local` (keep them commented in `.env.example` only).
- ☐
  Add a `docs/deferred.md` note recording the PRD rationale: "BootPulse MVP has no image, avatar-upload, or media-transformation requirement."

## 4.2 Gemini API — Confirmed Out of Scope

- ☐
  **N/A — CONFIRMED:** Do **not** install `@google/generative-ai` or any Gemini SDK.
- ☐
  **N/A — CONFIRMED:** Do **not** create any AI summary route, prompt builder, or generated-copy component.

- ☐
  **N/A — CONFIRMED:** Do **not** add `GEMINI_API_KEY` to `.env.local`.
- ☐
  Add a `docs/deferred.md` note recording: "AI-generated financial summaries are a Phase 2 feature; adds cost and trust risk before core accuracy is proven."

## 4.3 Stripe SDK & Server Client

- ☐
  Install `stripe` and `@stripe/stripe-js`.
- ☐
  Create `src/lib/stripe/client.ts` exporting a single server-side `Stripe` instance initialised with `STRIPE_SECRET_KEY` (test mode).
- ☐
  Confirm `src/lib/stripe/client.ts` has **no** `'use client'` directive.
- ☐
  Confirm `STRIPE_SECRET_KEY` is never imported into any Client Component.
- ☐
  Add an assertion that `STRIPE_SECRET_KEY` starts with `sk_test_` and throw on startup otherwise.
- ☐
  Add an assertion that `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` starts with `pk_test_`.

## 4.4 Tier → Price Mapping

- ☐
  Create `src/lib/stripe/prices.ts` on the server.
- ☐
  Map `starter` → `STRIPE_PRICE_STARTER_MONTHLY`.
- ☐
  Map `growth` → `STRIPE_PRICE_GROWTH_MONTHLY`.
- ☐
  Map `scale` → `STRIPE_PRICE_SCALE_MONTHLY`.
- ☐
  Export `resolvePriceId(tier: PlanTier): string` that throws on an unknown tier.
- ☐
  Confirm the mapping file is server-only and is **not** imported by any Client Component.
- ☐
  Confirm the client sends a **tier name**, never a `price_...` ID.
- ☐
  Confirm no `price_...` string literal is hardcoded anywhere in source.

## 4.5 Stripe Test-Mode Products

- ☐
  In the Stripe **test** dashboard, create product `BootPulse Starter` at `$12/month`.
- ☐
  Create product `BootPulse Growth` at `$29/month`.
- ☐
  Create product `BootPulse Scale` at `$59/month`.
- ☐
  Copy the three test-mode price IDs into `.env.local`.
- ☐
  Confirm the Stripe dashboard is in **test mode** and no live product exists.

## 4.6 Third-Party Boundary Documentation

- ☐
  Document in `docs/integrations.md` every third-party dependency: Neon, Clerk, Stripe.
- ☐
  For each, document: purpose, data sent, data received, region, retention implications.
- ☐
  List ImageKit and Gemini in the same document under **"Not integrated — Phase 1"** with the rationale.
- ☐
  Confirm the document contains **no** secret values.

## ✅ Verification Gate — Module 4

- ☐
  Run `npm run build` and confirm **zero TypeScript errors**.
- ☐
  Confirm `grep -r "imagekit" src/` returns no application imports.
- ☐
  Confirm `grep -r "generative-ai" src/` returns no imports.
- ☐
  Confirm `grep -rn "sk_test\|pk_test\|price_" src/ --exclude-dir=node_modules` returns **no hardcoded values** (only `process.env` references).
- ☐
  Confirm `src/lib/stripe/client.ts` throws when STRIPE_SECRET_KEY does not start with `sk_test_`.
- ☐
  Confirm `resolvePriceId('growth')` returns the value of `STRIPE_PRICE_GROWTH_MONTHLY` in a Node test.

---

# Module 5 — Core UI Workspace & Canvas Components

## 5.1 Design Tokens & Accessibility Primitives

- ☐
  Define the colour palette in `tailwind.config.ts` as named tokens (no ad-hoc hex values in components).
- ☐
  Verify every text/background token pair meets **≥ 4.5:1** contrast (≥ 3:1 for large text).
- ☐
  Define `--focus-ring` CSS custom property.
- ☐
  Add a global focus style: `outline: 2px solid var(--focus-ring); outline-offset: 2px;`.
- ☐
  Add a global rule prohibiting `outline: none` without an equivalent replacement.
- ☐
  Add `@media (prefers-reduced-motion: reduce)` global override disabling animations and transitions.
- ☐
  Install `@axe-core/react` (dev) and `axe-core` for automated checks.
- ☐
  Add a CI step running axe-core against every Phase 1 route.

## 5.2 Layout Shell & Navigation

- ☐
  Create `src/app/(app)/layout.tsx` as the authenticated shell.
- ☐
  Implement a 12-column grid with max content width `1440px`.
- ☐
  Implement a fixed `240px` sidebar.
- ☐
  Add the sidebar navigation: Dashboard, Cash Flow, Scenarios, Decisions, Milestones, Notes, Settings.
- ☐
  Implement `BusinessSwitcher` in the sidebar header.
- ☐
  Add a "New business" button in the switcher, gated by `assertWithinTier`.
- ☐
  Add landmarks: `<header>`, `<nav>`, `<main>`, `<aside>`.
- ☐
  Confirm each page renders exactly one `<h1>`.

## 5.3 Marketing Surfaces

- ☐
  Create `src/app/page.tsx` with `HeroSection` (`variant: "landing"`).
- ☐
  Add the primary CTA "Start free trial" linking to `/sign-up`.
- ☐
  Add a product screenshot placeholder (no ImageKit integration).
- ☐
  Create `src/app/pricing/page.tsx` rendering `PricingTable`.
- ☐
  Implement `PricingTable` with three tiers, a feature matrix, and a `currentTier` marker.
- ☐
  Create `src/app/legal/privacy/page.tsx` with the privacy notice including the **PICS disclaimer**.
- ☐
  Create `src/app/legal/terms/page.tsx` with the terms of service.
- ☐
  Confirm the privacy page states purpose of collection, transferee classes (Clerk, Neon, Stripe), retention period, and access/correction rights.

## 5.4 Workspace Dashboard

- ☐
  Create `src/app/(app)/businesses/[businessId]/page.tsx` rendering `WorkspaceDashboard`.
- ☐
  Implement `RunwayHero` showing runway months, zero-cash month, and current cash.
- ☐
  Implement `FormulaDisclosure` as a reusable expandable component.
- ☐
  Wire `RunwayHero`'s "Show the maths" to `FormulaDisclosure` using `formulaTrace` from the calc engine.
- ☐
  Implement `RunwayWarningBanner` with `severity: 'critical' | 'warning' | 'healthy'`.
- ☐
  Confirm the banner conveys severity via **icon + text label**, not colour alone.
- ☐
  Make the banner dismissible for the session only (no persistence).
- ☐
  Add a mini cash-flow chart with an adjacent accessible data table; mark the chart `aria-hidden`.
- ☐
  Add a "recent activity" list showing the last 5 decisions/notes.

## 5.5 Cash-Flow Table

- ☐
  Create `src/app/(app)/businesses/[businessId]/cash-flow/page.tsx`.
- ☐
  Implement `CashFlowTable` with month columns and a **sticky first column**.
- ☐
  Make the table header sticky on vertical scroll.
- ☐
  Render rows: opening cash, revenue total, expense total, net cash flow, closing cash.
- ☐
  Use real `<table>`, `<thead>`, `<tbody>`, and `<th scope="col">` / `<th scope="row">` markup.
- ☐
  Add a totals row.
- ☐
  Implement inline cell editing with `Enter` to commit and `Escape` to revert.
- ☐
  Implement `Tab` / `Shift+Tab` cell-to-cell keyboard navigation.
- ☐
  Implement bulk month fill for a selected row across a month range.
- ☐
  Confirm editing a cell recalculates runway and zero-cash month in **under 300ms**.
- ☐
  Enforce `tierHorizon` — months beyond the tier limit render read-only with a `TierLimitCallout`.
- ☐
  Implement CSV export at `GET /api/businesses/[id]/export` returning monthly summaries.
- ☐
  Confirm the CSV export route re-authenticates and re-authorises before responding.
- ☐
  Confirm the table container scrolls horizontally without the page scrolling horizontally at ≥ 1280px.

## 5.6 Scenario Planning

- ☐
  Create `src/app/(app)/businesses/[businessId]/scenarios/page.tsx`.
- ☐
  Implement `ScenarioComparisonTable` with baseline vs up to N scenarios.
- ☐
  Add a per-metric rows layout (runway, zero-cash month, closing cash at horizon) with a delta column.

- ☐
  Highlight the runway delta with an icon + text, not colour alone.
- ☐
  Implement "Clone to scenario" from the baseline.
- ☐
  Enforce `maxScenarios` from the tier map and render `TierLimitCallout` when exceeded.
- ☐
  Create `src/app/(app)/businesses/[businessId]/scenarios/[scenarioId]/page.tsx`.
- ☐
  Implement `ScenarioAssumptionEditor` with add/edit/delete rows.
- ☐
  Support assumption types: `add_expense`, `remove_expense`, `change_revenue`, `one_off_cost`, `one_off_gain`.
- ☐
  Support `startMonth`, `endMonth`, `amountMinor`, and `isEnabled` per assumption.
- ☐
  Recompute and persist `monthly_summaries` for the scenario on every assumption change.
- ☐
  Confirm the baseline scenario cannot be deleted or converted to a non-baseline kind.

## 5.7 Decision Log

- ☐
  Create `src/app/(app)/businesses/[businessId]/decisions/page.tsx`.
- ☐
  Implement `DecisionLog` as a reverse-chronological list.
- ☐
  Show title, status chip, linked scenario, and estimated monthly impact per entry.
- ☐
  Implement create / edit / status change (`considering`, `committed`, `rejected`, `revisited`).
- ☐
  Set `decidedAt` when status transitions to `committed` or `rejected`.
- ☐
  Allow linking a decision to a scenario via a searchable select.

## 5.8 Milestone Timeline

- ☐
  Create `src/app/(app)/businesses/[businessId]/milestones/page.tsx`.
- ☐
  Implement `MilestoneTimeline` as a horizontal dated timeline.
- ☐
  Render distinct past / today / future states with text labels, not colour alone.

- ☐
  Implement create / edit / state change (`planned`, `achieved`, `missed`).
- ☐
  Confirm the timeline is keyboard navigable and each milestone is focusable.

## 5.9 Notes

- ☐
  Create `src/app/(app)/businesses/[businessId]/notes/page.tsx`.
- ☐
  Implement note creation attached to a business.
- ☐
  Implement optional attachment to a month.
- ☐
  Implement optional attachment to a decision.
- ☐
  Render notes with author and timestamp.

## 5.10 Business Settings

- ☐
  Create `src/app/(app)/businesses/[businessId]/settings/page.tsx`.
- ☐
  Implement category management (create, rename, reorder, toggle `isRecurring`).
- ☐
  Implement business profile editing (name, currency, starting cash, start month).
- ☐
  Implement editable `burnWindowMonths`, `criticalThresholdMonths`, `warningThresholdMonths`.
- ☐
  Add a danger-zone "Delete business" action with a typed confirmation dialog.
- ☐
  Confirm deletion cascades through all tenant tables.

## 5.11 Account Settings

- ☐
  Create `src/app/(app)/settings/page.tsx`.
- ☐
  Display the consent record with `consentPrivacyAt`, `consentTermsAt`, and `marketingOptIn`.
- ☐
  Implement one-click **JSON export** of all user data.
- ☐

Implement one-click **CSV export** of all user data.

- ☐
  Implement account deletion with a typed confirmation.
- ☐
  On deletion: remove all tenant rows via cascade and request Stripe test-customer deletion.
- ☐
  Log the deletion request with timestamp and actor ID.

## 5.12 Responsive Behaviour

- ☐
  At ≥ 1280px: full editing, no horizontal page scroll.
- ☐
  Between 1024px and 1279px: full layout, wide tables scroll inside their own container.
- ☐
  Below 1024px: read-only summary view with headline runway figures and a notice that detailed planning requires a larger screen.
- ☐
  Below 768px: informational page only — no workspace editing controls rendered.
- ☐
  Confirm no editing affordance is reachable below 1024px.

## ✅ Verification Gate — Module 5

- ☐
  Run `npm run build` and confirm **zero TypeScript errors**.
- ☐
  Run `npm run lint` and confirm zero errors.
- ☐
  Run axe-core against `/`, `/pricing`, `/app/businesses/[id]`, `/cash-flow`, `/scenarios`, `/decisions`, `/milestones`, `/notes`, `/settings` and confirm **zero critical or serious** violations.
- ☐
  Manually verify every focusable element renders a **2px solid** focus ring.
- ☐
  Manually verify all body text meets **≥ 4.5:1** contrast.
- ☐
  Complete the full happy path: sign up → onboard → view dashboard → edit a cash-flow cell → clone to scenario → compare → record decision — and confirm it completes in **under 3 minutes**.
- ☐
  Confirm every displayed financial figure exposes a `FormulaDisclosure`.
- ☐
  Confirm the runway recalculates in under 300ms after a cell edit.

- ☐
  Run the keyboard-only pass: `Tab`, `Enter`, `Escape`, arrow keys operate the cash-flow table and scenario editor with no traps.

---

# Module 6 — Billing Integration, Webhooks, & Pre-Launch Audits

## 6.1 Checkout Session Route

- ☐
  Create `src/app/api/checkout/route.ts` handling `POST`.
- ☐
  Re-authenticate the Clerk session; reject with `401` if absent.
- ☐
  Validate the request body with Zod — accept a **tier name**, reject any `price_...` input.
- ☐
  Re-authorise `businessId` ownership via `requireBusinessAccess`.
- ☐
  Resolve the price ID server-side via `resolvePriceId(tier)`.
- ☐
  Reuse an existing `stripeCustomerId` or create a new Stripe **test** customer.
- ☐
  Create the Checkout Session with `mode: 'subscription'`.
- ☐
  Set `client_reference_id` to the internal user UUID.
- ☐
  Set `metadata: { user_id, business_id, target_tier }`.
- ☐
  Set `success_url` to `${APP_URL}/billing?checkout=success&session_id={CHECKOUT_SESSION_ID}`.
- ☐
  Set `cancel_url` to `${APP_URL}/billing?checkout=cancelled`.
- ☐
  Set `allow_promotion_codes: true`.
- ☐
  Return the Checkout Session URL to the client.
- ☐
  Add rate limiting to this route.
- ☐
  Confirm the route never returns or logs the secret key.

## 6.2 Billing Portal Route

- ☐
  Create `src/app/api/billing/portal/route.ts` handling POST.
- ☐
  Re-authenticate and re-authorise before creating the session.
- ☐
  Create a Stripe Billing Portal session for the stored test customer.
- ☐
  Return the portal URL.

## 6.3 Stripe Webhook Listener

- ☐
  Create `src/app/api/webhooks/stripe/route.ts` handling POST.
- ☐
  Read the **raw** request body before any parsing.
- ☐
  Verify the signature with `stripe.webhooks.constructEvent(rawBody, signature, STRIPE_WEBHOOK_SECRET)`.
- ☐
  Reject with HTTP `400` on verification failure.
- ☐
  Record `event.id` in `processed_webhook_events` and exit early on replay (**idempotency**).
- ☐
  Handle `checkout.session.completed`: resolve `client_reference_id` → user, persist `stripe_customer_id` and `stripe_subscription_id`, set `plan_tier` from `metadata.target_tier`, set status to `active` or `trialing`.
- ☐
  Handle `customer.subscription.created`: upsert subscription record, sync tier and period end.
- ☐
  Handle `customer.subscription.updated`: re-sync tier, status, and `current_period_end`.
- ☐
  Handle `customer.subscription.updated` **downgrade path**: mark data above the new limit read-only — **never delete**.
- ☐
  Handle `customer.subscription.deleted`: set status `canceled`, revert to `starter` limits, keep all data.
- ☐
  Handle `invoice.payment_failed`: set status `past_due`, surface an in-app banner, retain entitlements for a 7-day grace period.
- ☐
  Handle `invoice.payment_succeeded`: clear `past_due`, extend `current_period_end`.
- ☐

Return `200` quickly; queue any long work rather than awaiting it inline.

- ☐
  Reject any non-test-mode event.
- ☐
  Add rate limiting to this route.
- ☐
  Confirm **entitlements are webhook-authoritative** — `?checkout=success` never unlocks a tier.

## 6.4 Billing Page

- ☐
  Create `src/app/(app)/billing/page.tsx`.
- ☐
  Display the current plan and `subscription_status`.
- ☐
  Display usage vs limits (businesses, forecast horizon, scenarios).
- ☐
  Render `PricingTable` with the current tier marked.
- ☐
  Add the "Upgrade" button calling `POST /api/checkout`.
- ☐
  Add the "Manage billing" button calling `POST /api/billing/portal`.
- ☐
  Render a "finalising your plan" state when `?checkout=success` is present but the webhook has not yet landed.
- ☐
  Render a neutral no-op state for `?checkout=cancelled`.
- ☐
  Render `TierLimitCallout` on every gated surface linking back to `/billing`.

## 6.5 Compliance Implementation

- ☐
  Add the **PICS disclaimer** to `/legal/privacy` with PDPO-specific wording (not generic GDPR boilerplate).
- ☐
  Add the consent checkbox at sign-up for privacy + terms.
- ☐
  Add a **separate, unticked-by-default** marketing consent checkbox.
- ☐
  Confirm both consent timestamps are written server-side only.
- ☐
  Confirm consent events are append-only and never back-dated.

- ☐
  Document retention: active data while the account exists; deletion within 30 days; backups expire within 90 days.
- ☐
  Document the breach-notification procedure with the GDPR 72-hour and PDPO prompt-notification pathways.
- ☐
  Document Neon and Clerk regions and the cross-border transfer safeguard in the privacy notice.
- ☐
  Confirm no bank credentials, ID documents, or card data are stored (Stripe Checkout hosts the card form).

## 6.6 Application Security Hardening

- ☐
  Configure `Content-Security-Policy` in `next.config.ts`.
- ☐
  Configure `Strict-Transport-Security` in `next.config.ts`.
- ☐
  Configure `X-Content-Type-Options: nosniff`.
- ☐
  Configure `Referrer-Policy`.
- ☐
  Configure `Permissions-Policy`.
- ☐
  Add Zod validation to **every** Server Action and Route Handler at the trust boundary.
- ☐
  Confirm the webhook payload is validated **after** signature verification only.
- ☐
  Confirm all queries go through Drizzle's parameterised builder; raw SQL is limited to the reviewed RLS helpers.
- ☐
  Add an audit log entry for tier changes, consent events, and account deletions with timestamps and actor IDs.
- ☐
  Confirm the runtime DB role is not a superuser and does not hold BYPASSRLS.
- ☐
  Run `gitleaks detect` and confirm zero findings.

## 6.7 Accessibility Audit

- ☐
  Run axe-core across all Phase 1 routes and confirm zero critical/serious violations.
- ☐
  Verify all text meets ≥ 4.5:1 contrast (≥ 3:1 for large text).

- ☐
  Verify all focusable elements render `outline: 2px solid` with a 2px offset.
- ☐
  Verify focus is never obscured by sticky headers or overlays (WCAG 2.4.11).
- ☐
  Verify interactive targets are ≥ 24×24 CSS px; primary actions ≥ 44×44 px (2.5.8).
- ☐
  Verify help affordances appear in the same relative location on every workspace page (2.4.13).
- ☐
  Verify previously entered values are auto-populated or selectable within the session (3.3.7).
- ☐
  Verify sign-in supports paste and password managers with no cognitive-function test (3.3.8).
- ☐
  Verify runway severity uses icon + text label, not colour alone.
- ☐
  Verify every chart has an accessible data table equivalent.
- ☐
  Verify `prefers-reduced-motion` disables all animation.
- ☐
  Verify form errors use `aria-describedby` and are announced via `role="alert"`.
- ☐
  Run a manual VoiceOver pass on the dashboard and cash-flow table.
- ☐
  Run a manual NVDA pass on the dashboard and cash-flow table.

## 6.8 End-to-End Acceptance Validation

- ☐
  **AC1:** New user signs up, completes onboarding, and sees a calculated runway figure in under 3 minutes.
- ☐
  **AC2:** Every displayed financial figure exposes its formula via `FormulaDisclosure`.
- ☐
  **AC3:** Editing any cash-flow cell recalculates runway and zero-cash month in under 300ms.
- ☐
  **AC4:** User can clone the baseline, add "hire a freelancer at $2,000/month from month 3", and see the runway delta side-by-side.
- ☐
  **AC5:** Tier limits block creation beyond the plan; upgrading via Stripe **test mode** lifts the limit **only after** the webhook is processed.
- ☐
  **AC6:** axe-core reports zero critical or serious violations on all Phase 1 routes.
- ☐
  **AC7:** All text meets ≥ 4.5:1 contrast; all focusable elements render a 2px solid focus ring.

- ☐
  **AC8:** Secret scanning passes with zero findings; no live Stripe keys exist in any environment.
- ☐
  **AC9:** Account deletion removes all tenant rows via cascade within 30 days.
- ☐
  **AC10:** All shipped demo content is fictional and uses reserved example domains and number ranges.

## 6.9 Stripe Test-Mode Rehearsal

- ☐
  Run a full upgrade from `starter` → `growth` using Stripe test card `4242 4242 4242 4242`.
- ☐
  Confirm the limit is **not** lifted on the redirect alone.
- ☐
  Confirm the limit lifts after the webhook is processed.
- ☐
  Run a downgrade `growth` → `starter` and confirm excess data becomes read-only and is **not deleted**.
- ☐
  Simulate `invoice.payment_failed` and confirm the `past_due` banner appears and entitlements survive 7 days.
- ☐
  Simulate `customer.subscription.deleted` and confirm reversion to starter limits with data intact.
- ☐
  Replay a webhook event and confirm it is ignored via the idempotency table.
- ☐
  Confirm no live-mode Stripe key exists in any environment.

## ✅ Verification Gate — Module 6 (Release Gate)

- ☐
  `npm run build` — zero errors.
- ☐
  `npm run lint` — zero errors.
- ☐
  `npm run typecheck` — zero errors.
- ☐
  Full unit + integration test suite — all pass.
- ☐
  axe-core — zero critical/serious violations on every route.
- ☐
  `gitleaks detect` — zero findings.

- ☐
  All 10 Phase 1 Acceptance Criteria verified and ticked.
- ☐
  Confirm **no** Phase 2 feature has been partially implemented anywhere in the codebase.

---

# Deferred — Phase 2 (Post-Launch Expansion)

> **Do not build.** These items are recorded for roadmap continuity only. If any of these appear in a Phase 1 pull request, reject it.

## Deferred Integrations

- ☐
  Banking / open-banking integrations (requires a regulated aggregator, real credentials, reconciliation logic).
- ☐
  Read-only transaction import feeding `revenue_items` / `expense_items`.
- ☐
  Payroll systems integration (high compliance surface, country-specific).
- ☐
  Advanced accounting — double-entry, ledgers, P&L / BS / CF statements.
- ☐
  Optional "accountant export" bridge as an alternative to full accounting.
- ☐
  Tax estimates (requires a per-region rule engine plus disclaimers).
- ☐
  Multi-currency support (requires an FX rate source, historical rates, and a conversion policy).
- ☐
  Add a `fx_rates` table and per-business `currency_code` conversion.
- ☐
  AI-generated financial summaries via **Gemini API**.
- ☐
  ImageKit / media pipeline — re-evaluate only if receipts or invoice attachments are added.

## Deferred Product Surfaces

- ☐
  Adviser client portal with invitation, granular permissions, and audit trail.
- ☐
  Read-only adviser link for the **Scale** tier.
- ☐
  Advanced forecasting models — seasonality, cohort, Monte Carlo.
- ☐

Layer advanced models on top of `scenario_assumptions`.

- ☐
  Invoicing.
- ☐
  Document uploads.
- ☐
  Mobile-native applications.
- ☐
  Support for real customer financial data (Phase 1 remains fictional demo data only).

---

**End of tasklist.md**